



HANDS-ON SEMINAR

コーディングエージェント 入門

“あなたのPCで動くAI”を、**無料**で体験する

● ● ● 今日やること

```
$ opencode
```

```
> ToDoアプリを作って
```

```
作成中: index.html ...
```

情報学科 研究室向け / 所要 約1時間 / お金はかかりません

> 今日のゴール

1

“何か”が分かる

コーディングエージェントとは何か、普段のAIチャットと何が違うのかを理解する。

2

自分のPCで動かす

エージェントを実際にインストールし、AIモデルにつないで起動するところまでやる。

3

アプリを作らせる

チャットにとどまらず、エージェントに ToDo アプリを作らせてブラウザで動かす。

> 今日の流れ

1

知る：エージェントって何？

座学でイメージをつかむ

～15分

2

準備：エージェントを入れる

OpenCode をインストール

～15分

3

つなぐ：AIモデルに接続

APIキーを作って /connect

～10分

4

作る：ToDoアプリを生成

複数ファイルに分けて作らせる

～20分

> “コピペ”の、その先へ

ChatGPT や Claude に質問 → 返ってきた答えを自分でコピペ。

ふつうの AI チャット

- 質問すると文章を返してくれる
- コードも“テキストとして”もらえる
- でも…ファイルに保存するのは自分
- 実行・テスト・修正も全部自分



コーディングエージェント

コピペの“その先”を、AIが
あなたのPCの上で自分でやる。

> コーディングエージェントとは？

ひとことで言うと —— あなたのPCを操作できるAI。

1

ファイルを読む

プロジェクト全体を把握

2

ファイルを書く・直す

コードを実際に保存

3

コマンドを実行

テスト・サーバー起動も

4

エラーを読んで直す

結果を見て自分で修正

Webチャットとの一番の違いは“**手元のPCを直接動かせる**”こと。だから実際にファイルができて、その場で動かせる。

> 代表的なエージェント

Claude Code

Anthropic

高性能で人気。CLIから本格的な開発。基本は有料。

Codex CLI

OpenAI

OpenAI のCLIエージェント。
ChatGPTの延長で使える。

OpenCode

オープンソース

好きなモデルに接続できる。無料モデルでも動く。

どれも“ターミナルで動くエージェント”で、使う感覚はほぼ同じ。今日は誰でも無料で試せる OpenCode を使います。

> なぜ“ターミナル”で動くの？

ターミナル = PCに文字で命令する場所

- ファイル操作もコマンド実行も、ここから全部できる
- エージェントの“手”として一番都合がいい
- ボタンを探すより速く・自動で動かせる

こんな世界です

```
$ ls
index.html  README.md

$ python3 -m http.server
# → localhost:8000 で確認

$ opencode
> ToDoアプリを作って
  作成中: src/app.js ...
```

> 今日のツール：OpenCode

オープンソース

無料で使える。ターミナルで動くエージェント。

モデルが自由

OpenAI / Anthropic / OpenRouter など好きなモデルに接続。

なぜ今日これ？

誰でも無料で動かせるから。操作の感覚は他ツールと同じ。

TUI イメージ

```
$ opencode
```

OpenCode を起動しました

```
> _
```

ここに指示を書く

`/connect` でモデルに接続

> なぜ OpenRouter ?

1

1つのキーで多数のモデル

OpenRouter のAPIキー1本で、いろんなモデルを切り替えて使える。

2

無料モデルがある

今日はここを使う。料金は input / output とともに \$0。

3

普段のquotaを使わない

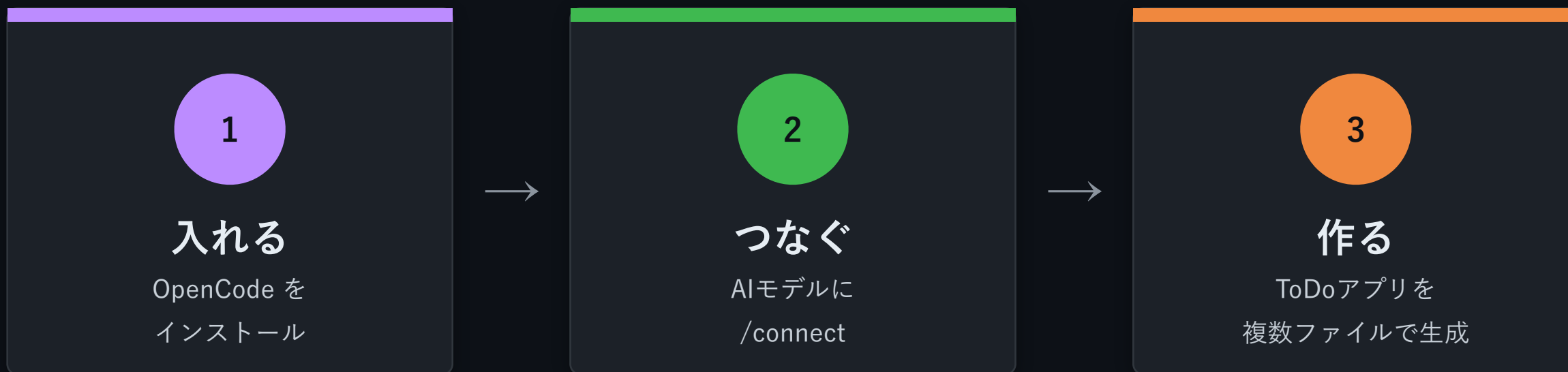
ChatGPT等の課金枠を消費しないので、気兼ねなく試せる。

4

Free Models Router

空いている無料モデルへ自動でつないでくれる。

> 今日やること（全体図）



この3ステップを、これから順番にやっていきます。

> STEP 0 —— ターミナルを開く

macOS

- 「ターミナル.app」を開く
- Spotlight で“ターミナル” 検索でもOK

Windows

- WSL の Ubuntu ターミナルを開く
- PowerShell ではない点に注意

⚠ **Windows の人へ：** この先のコマンドはすべて Ubuntu のターミナル内で実行します。

● ● ● 動作確認

```
$ pwd # 今どこにいるか確認
```

> STEP 1 —— OpenCode を入れる

macOS + Homebrew

```
# Homebrew を更新  
$ brew update  
  
# OpenCode を入れる  
$ brew install \\\n    anomalyco/tap/opencode  
  
$ opencode --version
```

Windows (WSL Ubuntu)

```
# 必要なツール  
$ sudo apt update  
$ sudo apt install -y \\\n    curl ca-certificates  
  
# OpenCode を入れる  
$ curl -fsSL \\\n    https://opencode.ai/install | bash
```

インストール手順だけはOSで違いますが、この先は Mac も WSL も共通です。

> STEP 2 —— OpenRouter API キーを作る

1

サイトを開く

openrouter.ai/keys にアクセス

2

サインイン

OpenRouter アカウントでログイン（無料）

3

キーを作成

「Create Key」を押す

4

控える

sk-or-... で始まるキーをコピーしておく

⚠ キーの取り扱い

- パスワードと同じ秘密情報
- ファイルやチャットに貼らない
- 人に見せない
- OpenCode が安全に保存する

> STEP 3 —— フォルダを作って起動する

作業用のフォルダを作り、その中で OpenCode を起動します。

● ● ● フォルダ作成 → 起動

作業フォルダを作って入る

```
$ mkdir todo-app
```

```
$ cd todo-app
```

OpenCode を起動

```
$ opencode
```

フォルダ名は自由

好きな名前でもOK（例: todo-app）。これから作るファイルは、このフォルダの中にできていきます。

> STEP 4 —— /connect で接続する

1

/connect と入力

TUIの入力欄に打って Enter

2

OpenRouter を選ぶ

provider 一覧から選択

3

API キーを貼る

STEP 2 で控えたキーを貼り付け

入力イメージ

```
> /connect
```

```
? Provider
```

```
> OpenRouter
```

```
? API Key
```

```
> sk-or-.....
```

```
✓ 接続しました
```

> STEP 5 —— Free Models Router を選ぶ

モデル選択画面で **Free Models Router** を選びます。

● ● ● モデルを探す

見つからなければ検索欄に：

Free Models Router

と入力して絞り込む

動作確認

Reply with OK only. と送って **OK** が返れば、接続成功です 🎉

> つかいかた（基本）

指示を書いて Enter

やってほしいことを日本語で書いて送るだけ。

様子が表示される

ファイルを作る・編集する過程がそのまま見える。

確認しながら進む

変更内容を見て、続けて指示を重ねていける。

やりとりの例

> ミニToDoアプリを作って

● 作成中

+ index.html

+ style.css

+ script.js

✓ ブラウザで表示確認

✓ 追加・完了・削除を確認

> STEP 6 —— 作るものを決める

今日は、ブラウザだけで動く ToDo ミニアプリを作ります。

追加・完了・削除

基本操作がその場で試せる

件数が見える

残り件数と完了件数を表示

閉じても残る

localStorage に保存

役割ごとに分ける

画面・見た目・動きを読みやすく



> このプロンプトを貼る

● ● ● OpenCode に貼り付ける指示

ブラウザで動くミニToDoアプリを作ってください。

- タスクの追加、完了切り替え、削除ができる
- 残り件数と完了件数を表示する
- localStorage に保存し、再読み込み後も残る
- 初回表示時はサンプルタスクを3件入れる
- スマホでもPCでも見やすいカード風の見た目

実装条件:

- HTML / CSS / JavaScript を役割ごとに分ける
- 作ったファイルと確認手順を最後に教える

確認すること:

- 追加・完了切り替え・削除・再読み込み後の保存

python3 -m http.server 8000 で起動して確認し、
問題があれば修正してください。

ポイント： 完成条件まで書くと無料モデルでも安定。役割と成功条件を渡す。

> ブラウザで動かしてみる

生成された index.html をブラウザで開いて、実際に操作します。

ローカルサーバーを起動

```
# プロジェクトのフォルダで  
$ python3 -m http.server  
  
# ブラウザで開く  
→ http://localhost:8000
```

エージェントに頼んでもOK:

「このフォルダをブラウザで開いて確認して」と言えば、確認まで任せられます。



> エージェントへの指示ファイル

プロジェクトに“指示ファイル”を置くと、前提やルールを毎回伝えられます。

ツール	指示ファイル
OpenCode	AGENTS.md
Codex CLI	AGENTS.md
Claude Code	CLAUDE.md

名前は違ってても“前提を1度書いておく”という考え方は共通です。

● ● ● 書き方の例

```
# AGENTS.md
## ルール

- 返答は日本語で
- 小さく動くものから作る
- まずブラウザで動作確認する
```

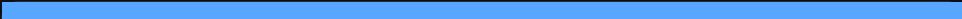
毎回同じ指示を書かずに済み、チームで前提もそろえられる。

> 無料枠の上限に注意

未課金アカウントの無料モデルには、OpenRouter の利用上限があります。


50

リクエスト / 日


20

リクエスト / 分

上限に当たったら、少し時間をおいてから再開しましょう。

> 今日できたこと



コーディングエージェントが何か理解した



OpenCode を自分のPCにインストールした



OpenRouter の無料モデルに接続した



ToDo ミニアプリを作らせて確認した

> もっと先へ

今日おぼえた“頼み方”と“指示ファイル”は、どのエージェントでもそのまま使えます。

他のエージェントを試す

Claude Code・Codex CLI など。今日の知識がそのまま活か
きる。

高性能モデルで本格開発

有料モデルにすると精度・速度がぐっと上がる。

Webアプリに挑戦

Node を入れて Vite などでの本格的なアプリへ。

ふだんの作業に使う

調べもの・修正・自動化など、研究や開発の相棒に。

> 参考リンク

OpenCode（今日のツール）

opencode.ai/docs

OpenRouter API キー

openrouter.ai/keys

Claude Code

claude.com/claude-code

Codex CLI

github.com/openai/codex

THANK YOU

おつかれさまでした

今日の続きは、あなたのPCの中で。

● ● ● Happy hacking

\$ `opencode`

> 次は何を作ろうか？